



// Building a JavaScript Runtime for the Desktop

DouJS

DouSec

by Shikr

Table of Contents

1. Introduction	4
2. Architecture	5
2.1. Overview	5
2.2. JavaScript Engine	5
2.3. Module System	5
2.4. Commands	6
2.5. Event Loop	6
3. Standard Library	7
4. User Interface	8
4.1. Rendering Engine	8
4.2. Input Handling	8
4.3. Text Rendering	8
4.4. Layout Engine	8
4.5. Event System	8
4.6. Reactivity	9
4.7. Accessibility	9
4.8. Styling	9
5. Rust Bridge	10
5.1. TypeScript Integration	10





1. Introduction

For now, this is a placeholder for the introduction section of the DouJS project. The goal of this project is to create a JavaScript runtime that can be used on desktop environments, providing developers with a powerful tool for building cross-platform applications.

2. Architecture

2.1. Overview

The architecture of DouJS is designed to be modular and extensible, allowing developers to easily add new features and capabilities. The core components of the runtime include:

- **JavaScript Engine:** The heart of the runtime, responsible for executing JavaScript code.
- **Module System:** A system for managing and loading JavaScript modules, enabling code reuse and organization.
 - **Native APIs:** A set of APIs that provide access to native desktop functionality, such as file system access, window management, and more.
 - **Standard Library:** A collection of pre-built functions and objects that provide common functionality for JavaScript developers.
 - **Dependencies:** A system for managing and resolving dependencies between different modules and libraries.
- **Commands:** A tauri-based command system that allows developers to define native functions that can be called from JavaScript code, enabling seamless integration with the underlying operating system.
- **Event Loop:** A mechanism for handling asynchronous operations and managing the execution of JavaScript code in a non-blocking manner.
- **Event System:** A system for handling and dispatching events within the runtime, allowing for reactive programming and real-time updates.

! **Problem:** At the moment, the Event System is not fully planned out, and its implementation may change as the project progresses. The current design is focused on providing a flexible and efficient way to handle events, but further research and development will be needed to ensure that it meets the needs of developers.

2.2. JavaScript Engine

The JavaScript Engine is the core component of DouJS, responsible for interpreting and executing JavaScript code. It provides a fast and efficient execution environment that supports modern JavaScript features and APIs. The `App` structure will serve as the main entry point for the runtime, managing the lifecycle of the application and coordinating the various components of the architecture.

With the `App` structure, developers can easily create and manage their applications, leveraging the power of the JavaScript Engine to build rich and interactive desktop experiences. It also will allow to the developers to define their own modules and commands, enabling them to extend the functionality of the runtime and integrate with native desktop features.

In conclusion, the `App` structure will provide a simple and intuitive interface for developers to define and manage their own application components, while also allowing them to start writing JavaScript code that can interact with the underlying system. DouJS is not designed to be a full-fledged framework, but rather a runtime that provides the necessary tools and capabilities for developers to build their own applications.

2.3. Module System

DouJS aims to be a lightweight and efficient JavaScript runtime, to achieve this, it will not force developers to use all the features provided by the runtime. Instead, it will allow developers to define their own modules. The module system will handle the following types of modules:

- **Native Modules:** Modules defined directly in the runtime, providing access to native desktop functionality and APIs. These modules can be those from the standard library or custom modules defined by the developers and will be compiled into the binary of the runtime.
- **JavaScript Modules:** Modules written in JavaScript, allowing developers to create reusable code and organize their applications into logical components.
- **Third-Party Modules:** Modules developed by the community or external libraries, which can be easily integrated into DouJS applications.
- **Dynamic Modules:** Modules that can be loaded and unloaded at runtime, enabling developers to add or remove functionality as needed.

! **Problem:** Third-party are not fully planned out yet, but the goal is to provide a simple and efficient way for developers to integrate external libraries and modules into their DouJS applications, using NPM modules as inspiration.

! **Problem:** Dynamic modules are not fully planned out yet, but the goal is to provide a flexible and efficient way for developers to load and unload modules at runtime.

2.4. Commands

Commands are a key feature of DouJS, inspired by Tauri's command system. They allow developers to define native functions that can be called from JavaScript code avoiding to create a new module for each native function. Commands will be defined in the `App` structure and can be easily registered and invoked from JavaScript code. To provide data to the commands, developers can use the `Command` structure, which allows them to define input parameters that need to be defined in the `App` structure. An example of a command definition in Rust is as follows:

```
*[command(rename = "myCommand")]
async fn my_command(input: String, title: Data<String>) -> String {
    // Command implementation goes here
    format!("Hello, {}! Received input: {}", title, input)
}
```

And the corresponding JavaScript code to call the command would look like this:

```
const result = await myCommand({ input: "John Doe" });
console.log(result); // Output: Hello, World! Received input: John Doe
```

2.5. Event Loop

JavaScript is a single-threaded language, which means that it can only execute one task at a time. To handle asynchronous operations and manage the execution of JavaScript code in a non-blocking manner, DouJS will implement an event loop. The event loop will be responsible for managing the execution of tasks, handling events, and coordinating the various components of the runtime. When creating a window, we will have a problem with the event loop, because winit needs its own event loop to function properly. The plan is to integrate winit's event loop with the existing event loop of DouJS, ensuring that input events are processed correctly and efficiently. Another solution could be to use a separate thread for the winit event loop and just send messages to the main event loop, but this solution is not ideal because it could introduce additional complexity and potential performance issues. Por el momento, las tareas que se ejecutan en el event loop son las siguientes:

- **execute_jobs()**: Executes the jobs that have been scheduled in the event loop, ensuring that they are processed in a timely manner.
- **yield_now()**: tokio::task::yield_now() is used to yield the current task, allowing other tasks to be executed in the event loop. This helps to prevent blocking and ensures that the event loop remains responsive.



3. Standard Library

👉 **Note:** TODO: describe the standard library and its components, including the available functions and objects, as well as any additional features or capabilities provided by the library.

4. User Interface

One of the key features of DouJS is its ability to provide a user interface for desktop applications. The runtime will include a set of APIs and components that allow developers to create rich and interactive user interfaces using JavaScript. For now, the main idea is to provide a widget-based system inspired by GTK, using the signal-slot pattern to handle events and interactions. The ideal would be to develop a Toolkit UI independent of JavaScript, but that is easy to use from JavaScript, allowing developers to create native-looking applications with minimal effort.

4.1. Rendering Engine

The planned rendering engine for long-term is wgpu, but for now, the plan is to use a simpler rendering engine that can help to prototype the UI system and test the functionality of the runtime, femtovg will be used as a temporary solution. The rendering engine will be responsible for drawing the user interface components on the screen and handling the rendering pipeline. It will be designed to be flexible and extensible, enabling developers to create custom widgets and components as needed.

4.2. Input Handling

To create the window and handle input events, the plan is to use winit, which provides a cross-platform windowing and input handling solution. Winit will allow DouJS to create windows, manage input events such as keyboard and mouse interactions, and handle window resizing and other system events. Winit needs its own event loop to function properly, so the plan is to integrate it with the existing event loop of DouJS, ensuring that input events are processed correctly and efficiently. To abstract the input events and provide a consistent interface for the integration with the rest of the runtime, a custom input handling system will be developed. This system will only receive the input events from winit and will be responsible for dispatching them to the appropriate components and widgets within the user interface.

4.3. Text Rendering

The text rendering has been a common problem in this type of toolkits, we do not know the real problem until we start the implementation, but for now, cosmic-text is a library that we could take into account.

4.4. Layout Engine

The layout engine will be responsible for arranging and positioning the user interface components within the application window. It will provide a flexible and efficient way to define the layout of the UI, allowing developers to create responsive and adaptive interfaces that can adjust to different screen sizes and resolutions. The layout engine will support various layout strategies, such as grid-based layouts, flexbox-like layouts, and absolute positioning. This feature is not fully planned out yet, but we will explore existing layout engines and libraries like stretch, yoga, taffy, and others to determine the best approach for implementing the layout engine in DouJS.

4.5. Event System

The event system will be responsible for managing and dispatching events within the user interface. It will provide a mechanism for components to listen for and respond to various events, such as:

- **Hit testing:** Determining which component or widget is under the cursor or has been interacted with, allowing for accurate event handling and user interaction.
- **Drag & drop:** Implementing a system for handling drag and drop operations within the user interface.
- **Gestures:** Recognizing and responding to touch gestures, such as swipes, pinches, and taps, enabling developers to create touch-friendly applications.

4.6. Reactivity

The reactivity system will be responsible for managing the dynamic updates and state changes within the user interface. It will provide a mechanism for components to automatically update when their underlying data changes, ensuring that only the necessary parts of the UI are re-rendered. The batched updates will be handled by the event loop, but the reactivity system will be responsible to orchestrate the updates and prevent unnecessary re-renders.

4.7. Accessibility

The accessibility system will be responsible for ensuring that the user interface is accessible to all users, including those with disabilities. It will provide support for screen readers, keyboard navigation, and other accessibility features. `UIKit` is a library that we could take into account for this feature, but we will need to explore other options and determine the best approach for implementing accessibility in DouJS.

4.8. Styling

The styling system will be responsible for defining the visual appearance of the user interface components. It will provide a mechanism for developers to apply styles and themes to their applications, based on declarative styling. Also, it will provide support for default styles and themes, allowing developers to create visually appealing applications with minimal effort.

5. Rust Bridge

We will provide a Rust bridge that allows developers to write native Rust code and expose it to JavaScript, enabling them to a easiest way to declare JavaScript components in Rust than the current QuickJS bindings. Also, we want to enhance the hybrid approach, allowing developers to write parts of their applications in Rust and other parts in JavaScript, leveraging the strengths of both languages, prioritizing the development experience with a TypeScript integration.

5.1. TypeScript Integration

In the Rust side, we will provide macros that will generate TypeScript definitions for the Rust components, allowing developers to have type safety and autocompletion when working with the Rust bridge. Also, using the **OXC** project, we will provide TypeScript support in the JavaScript side, allowing developers to write their applications in TypeScript and take advantage of its features, such as static typing and advanced tooling.